Components

Component Docker

Code and Documentation by Avery Norris

One major and neccessary requirement of an **EC/ECS** (Entity Component System) is **Nested-Components**; In **Osmium's** own **ECS** we have two objects that need to be able to hold **Children-Components**

```
Scenes

Components
```

Originally, both classes carried methods to store **Components** as children; However this was extremely redundant, and **Component-Docker** was created to handle this in an organised fashion!

Purpose

Called a **Docker** for short; the **Component-Docker** carries **three main functionalities**

```
Storing Children Component

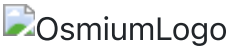
Providing an Interface to Modify Children

Transferring Events to the appropriate Components
```

- **Storing Children** is the **Main Job** of a Docker. Both Scenes and Components can host children. And the docker inheritance is what accommodates this. It uses a HashSet internally, and hashes Components in a list and two Dictionaries based on Priority, Type and Tags
- Osmium Provides an **Extensive Interface** to work with Components. You can check the API to see the MANY overloads for adding, getting, removing and moving Components provided.
- **Transferring Events** is the final usage for the **Docker**. When an Event is sent in **Osmium**, it is passed down between Dockers via ChainEvent(). Dockers call all the Dockers below themselves, and their inherited type. This is stopped if a Docker is a Component that is not enabled, or we reach the leaves of all branches in the **Component-Tree**.

End Of Documentation

Code and Documentation by Avery Norris


OsmiumLogo

Scene

In **Osmium**, The **Scene** is responsible for Containing all the "Action". It inherits a **ComponentDocker**; and carries many Components.

In Osmium, they are treated as **Even Entities** and can be loaded in parallel like so :

```
Osmium

    Loaded Scene A

    Loaded Scene B
```

Both Scene A and B will receive events independent of each other. It is also good to note that Components cannot exist at the **Scene-Level** (parallel of Scenes, like how A is to B). Components can only exist as a child of a Scene or a Child of a Component in a Scene.

Usage

Using a Scene is easy! Simply call **Osmium.AddScene(string name)** to create a new Scene! Or you can add your own with **Osmium.AddScene(Scene scene)**. The reason you might want to add your own is to allow for custom scene types. If you have seen Components, then you might know that Osmium **supports** external inheritance! It's as easy as inheriting Scene, and using AddScene() to make your own Custom Scene Code!

For further guidance, refer to the **API** documentation! With **Osmium** you are able to Add, Get, and Remove Scenes.

End Of Documentation

Code and Documentation by Avery Norris

